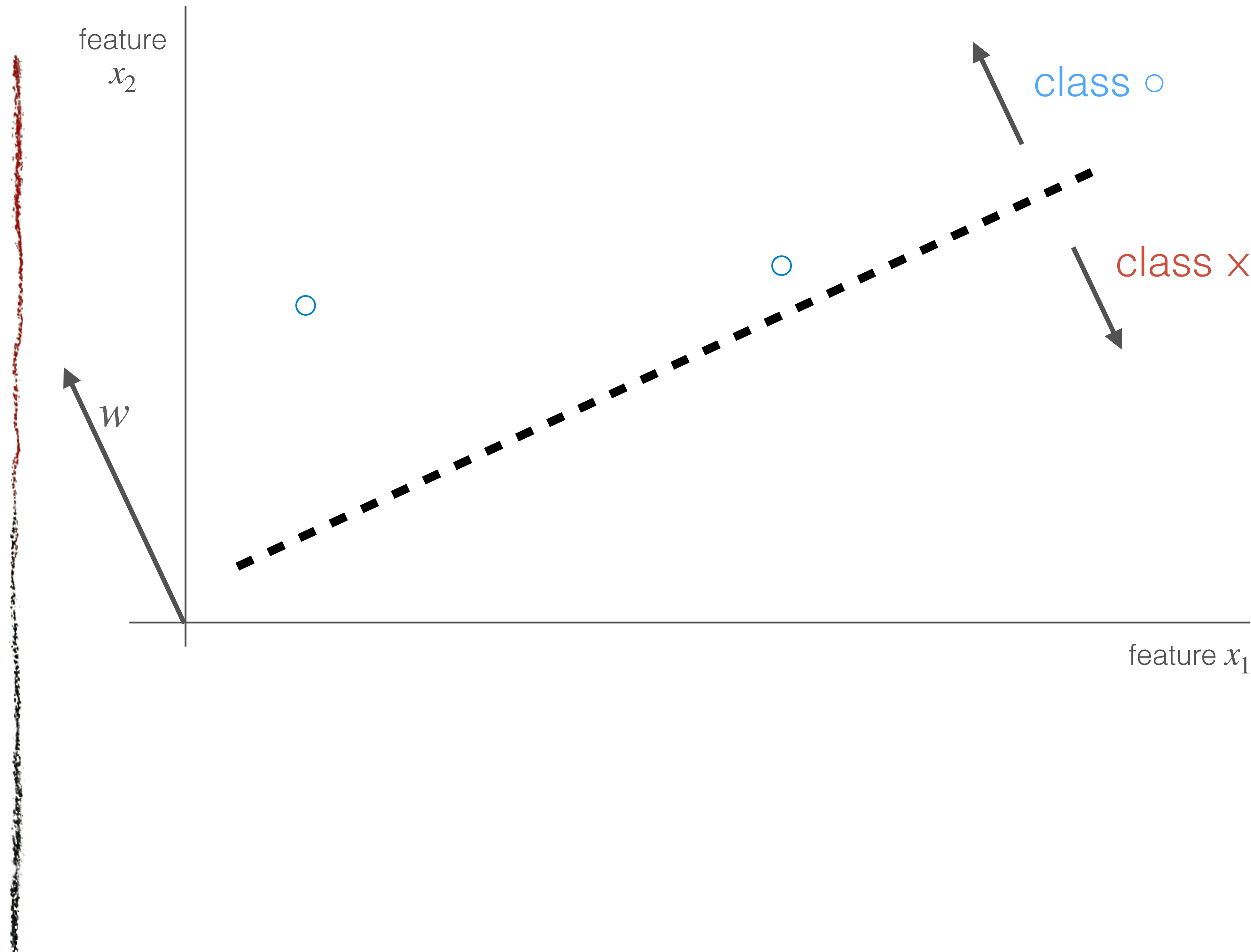


Linear classifiers

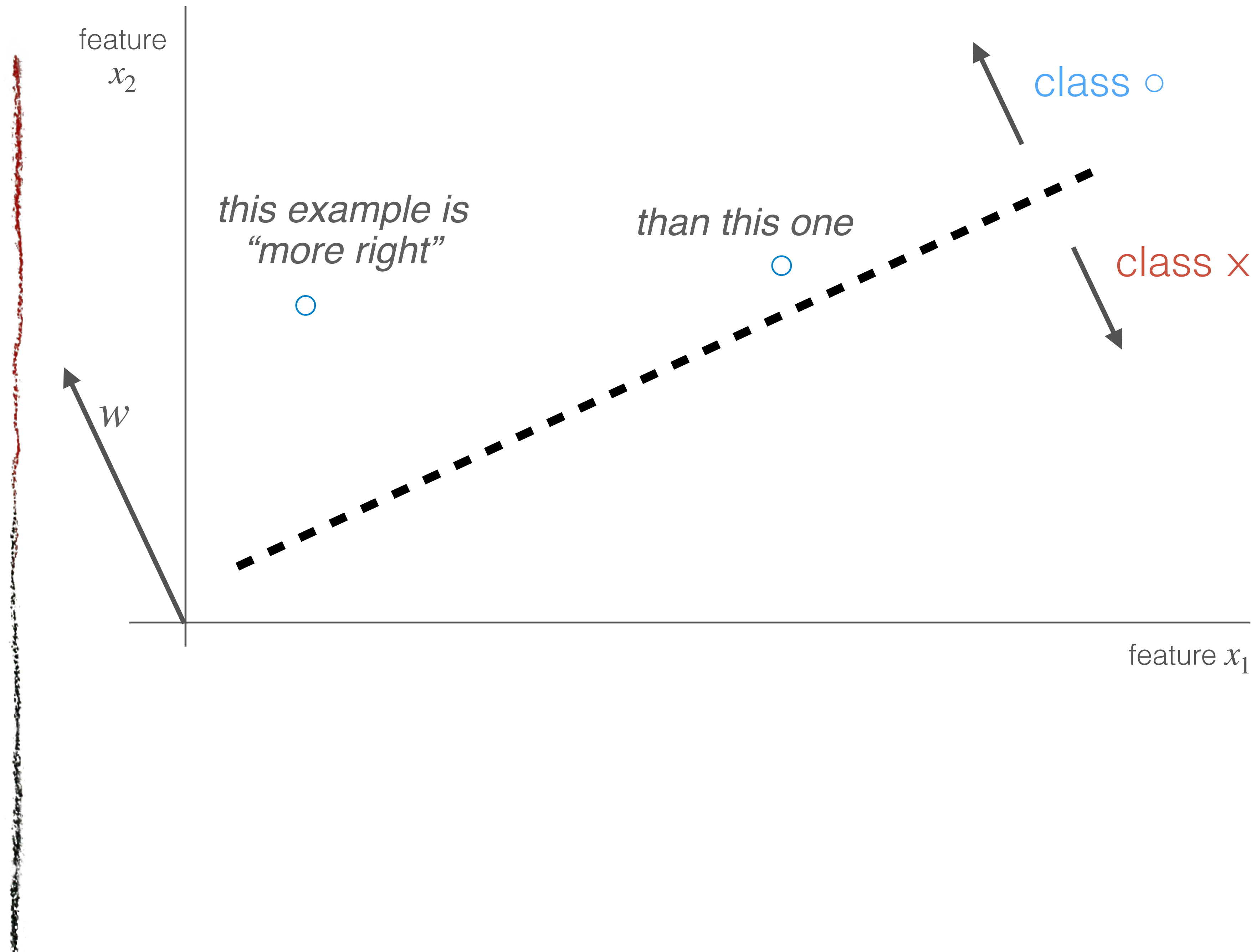


10-701 Introduction to Machine Learning
Geoff Gordon

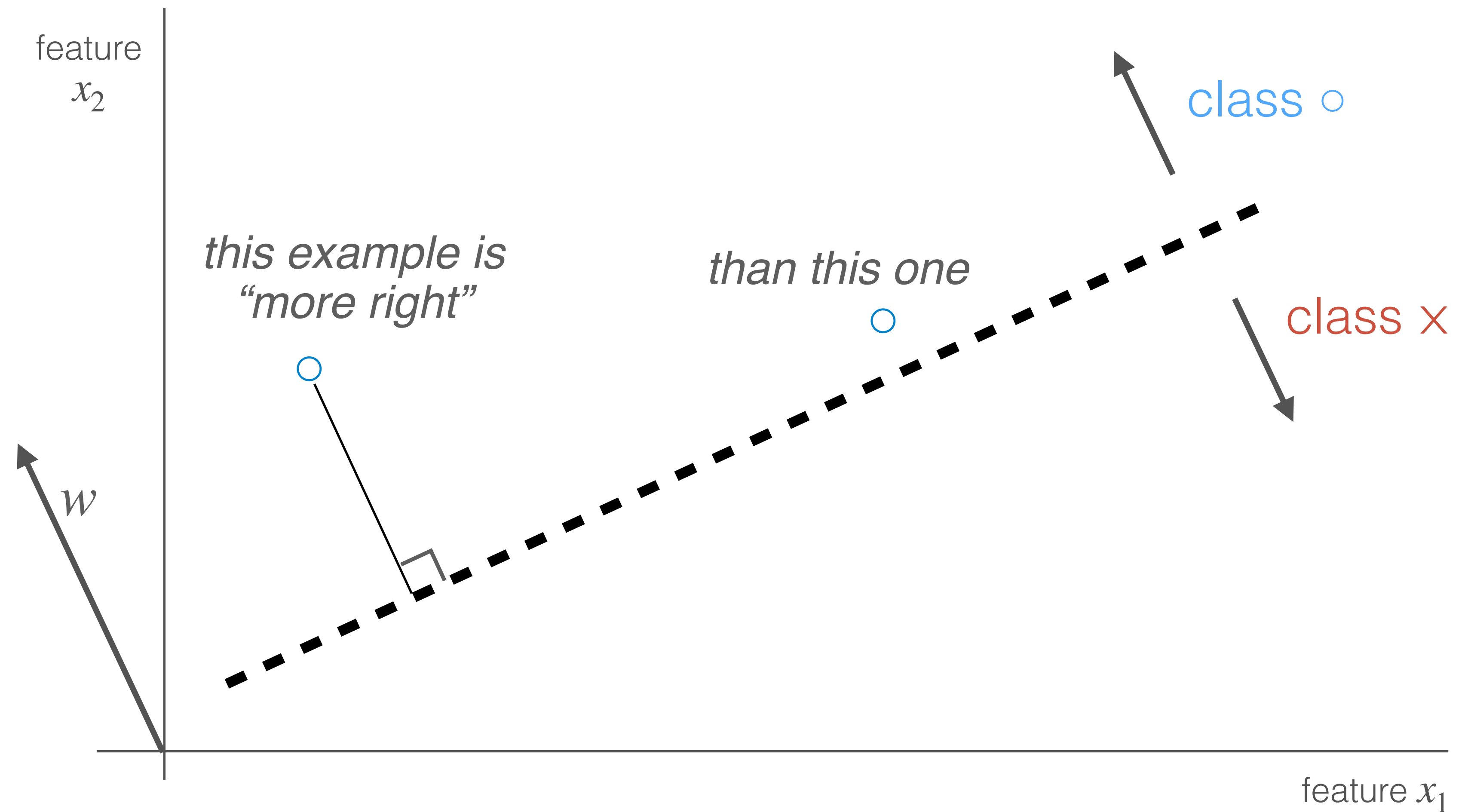
Margin



Margin



Margin



- Margin of example = $\perp$ distance to decision boundary

calculate: $w \cdot (x - \lambda \frac{w}{\|w\|}) + b = 0$

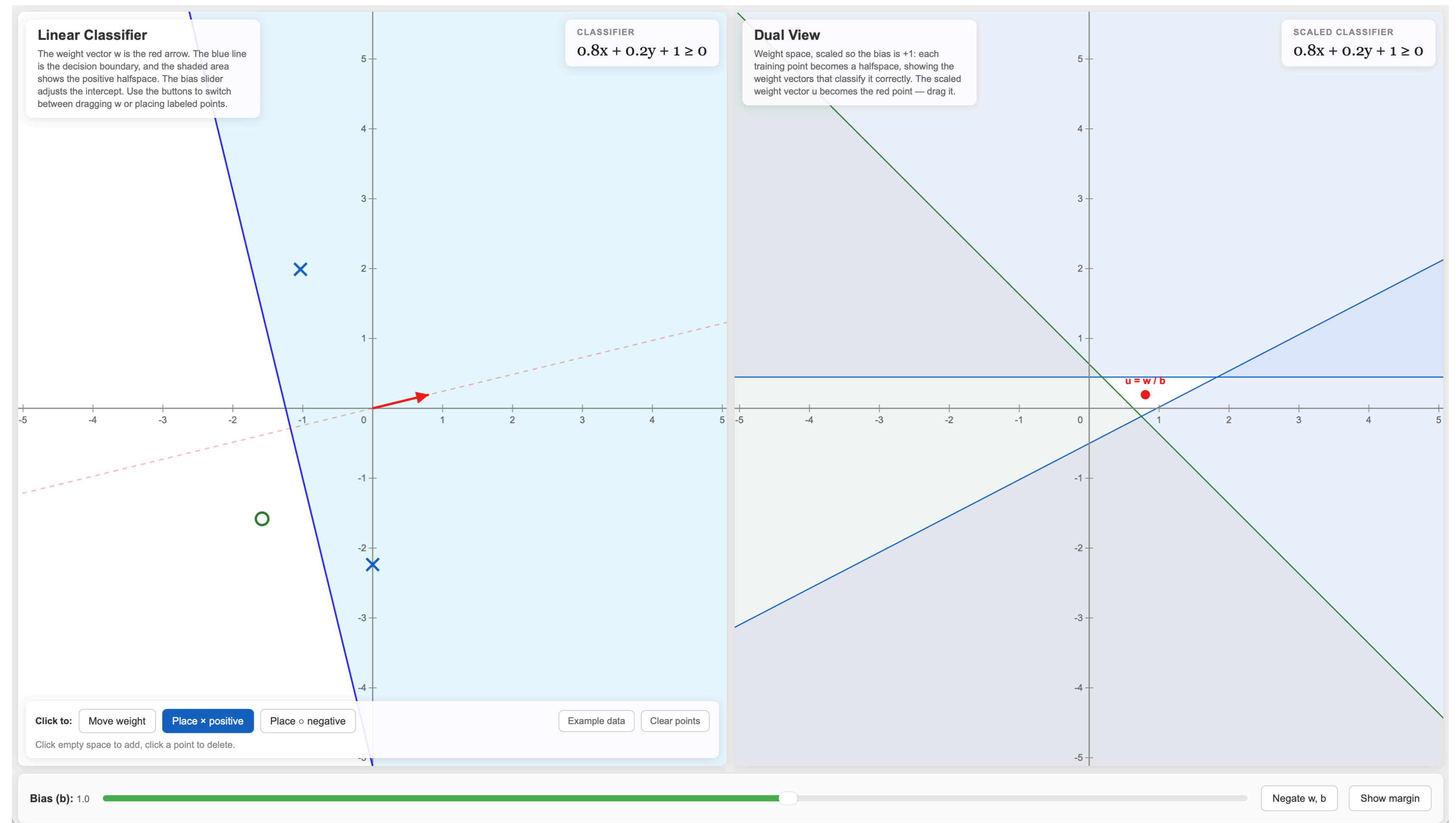
$$w \cdot x + b = \lambda \frac{w \cdot w}{\|w\|} = \lambda \|w\|$$

$$\frac{w \cdot x + b}{\|w\|} = \lambda$$

- Margin of dataset = smallest margin of any example

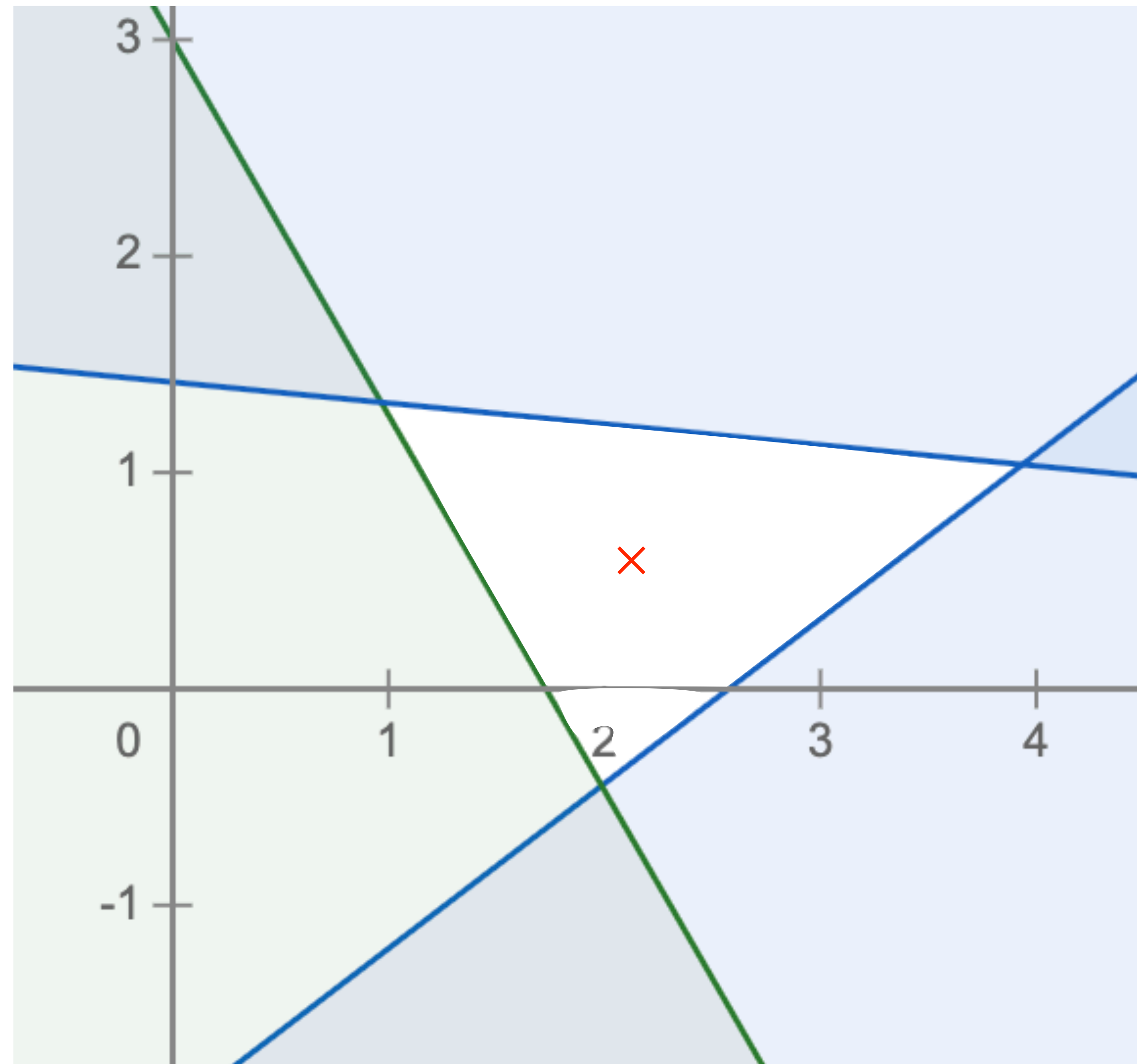
Margin geometry

interactive demo



- Good margin = weights near middle of version space

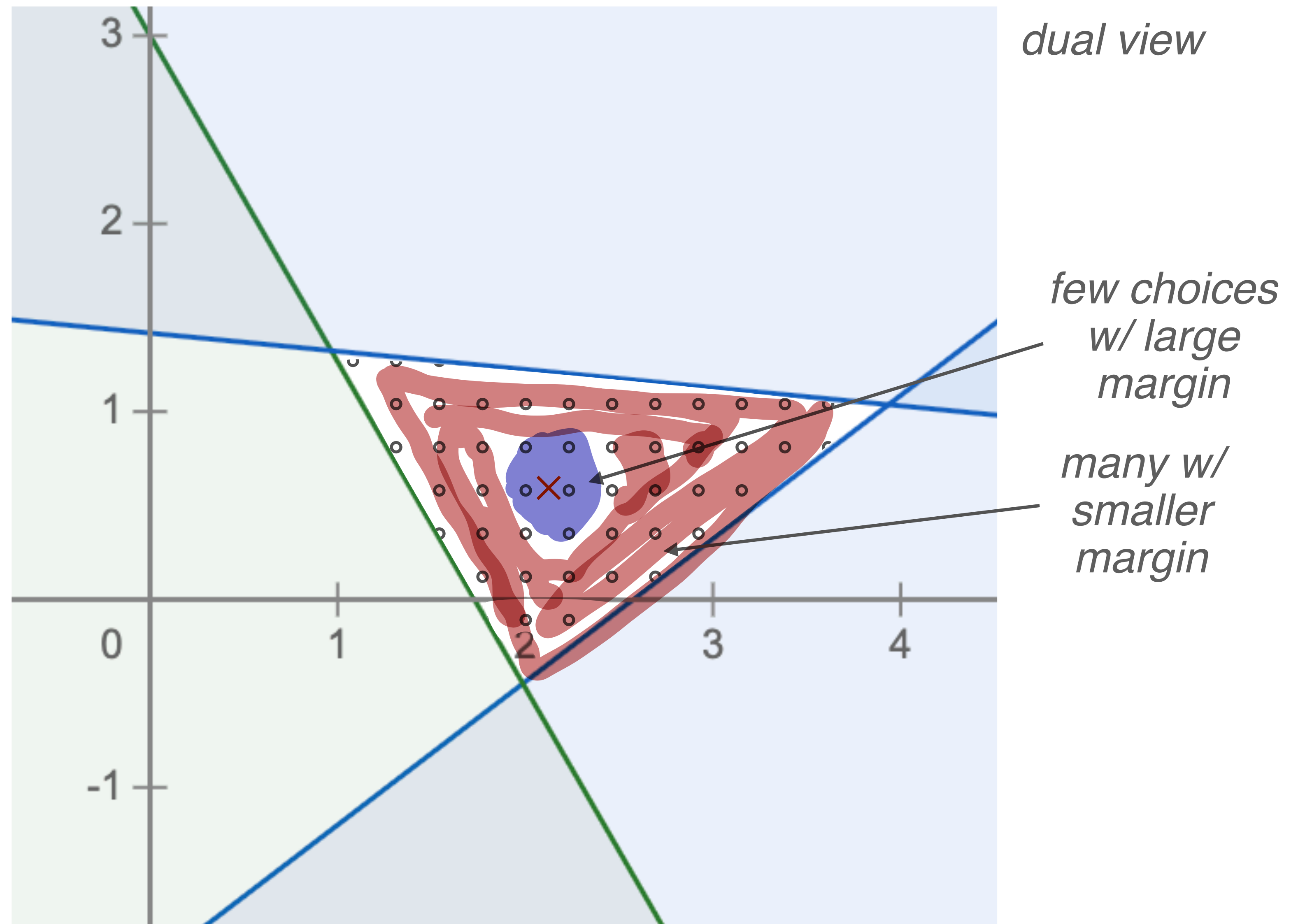
Complexity and generalization



dual view

- Require large margin $\rightarrow$ limit # of eligible hypotheses $\rightarrow$ hopefully better generalization

Complexity and generalization



- Require large margin $\rightarrow$ limit # of eligible hypotheses $\rightarrow$ hopefully better generalization

Perceptron mistake bound

*an example result
showing high margin →
good generalization*

- Suppose examples are (for all t)
 - ▶ bounded, $\|X^{(t)}\| \leq R$
 - ▶ separable with margin γ , $Y^{(t)}(w^* \cdot X^{(t)} + b) / \|w^*\| \geq \gamma$ for some unknown w^*
- Then perceptron makes at most $(R/\gamma)^2$ mistakes (ever)
- Generalization: if separable *after ignoring violations that sum to L* , we make at most $2L + (R/\gamma)^2$ mistakes
- Inductive bias of perceptron: examples have similar scale, are separable with high margin (and low violation)

Learning objectives— linear classifiers

- How does the decision boundary relate to the parameters w, b of a linear classifier?
- How do changes like scaling the weight vector, adjusting the bias, or negating w, b affect the decision boundary?
- What does the dual space for a linear classifier represent? How do changes in the examples or parameters affect it?
- What is effect of multiplying each constraint by label y ?
- What is the difference between the online and batch learning settings?
- What is a margin?
- How does the perceptron algorithm work?
- What is its inductive bias?

Logistic regression



10-701 Introduction to Machine Learning
Geoff Gordon

Classification

- Generative story:
 - ▶ $X^{(t)}$ comes from some unknown distribution (we aren't going to try to recover it)
 - ▶ compute $P(Y(t) = 1 \mid X^{(t)})$ as some function $f_{\theta}(X^{(t)})$
 - ▶ sample $Y^{(t)} \sim \text{Bernoulli}(\hat{Y}^{(t)})$ where $\hat{Y}^{(t)} = f_{\theta}(X^{(t)})$

Linear model for classification

- Linear model $\hat{Y}^{(t)} = w \cdot X^{(t)} + b$ is great for regression
- Use $\hat{Y}^{(t)} = f_{w,b}(X^{(t)}) = w \cdot X^{(t)} + b$ for classification?

No, outside $[0, 1]$

Bernoulli model

- Coin flip: $P(Y = H) = \hat{Y} = p$
- Likelihood:
 - ▶ factor of p if H, factor of $(1 - p)$ if T
 - ▶ concisely: $p^Y(1 - p)^{1-Y}$
- Log likelihood:

- ▶ $Y \ln p + (1 - Y) \ln(1 - p) = Y \ln \frac{p}{1-p} + \ln(1-p)$

Bernoulli model

called **log-odds**
or **logit**

- $L = -[Y \ln \frac{p}{1-p} + \ln(1-p)]$ define $z = \ln \frac{p}{1-p}$

► solve for p :

$$p = \sigma(z) \quad e^z = \frac{p}{1-p} \quad (1-p)e^z = p \quad e^z = p(1+e^z)$$

$$p = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}}$$

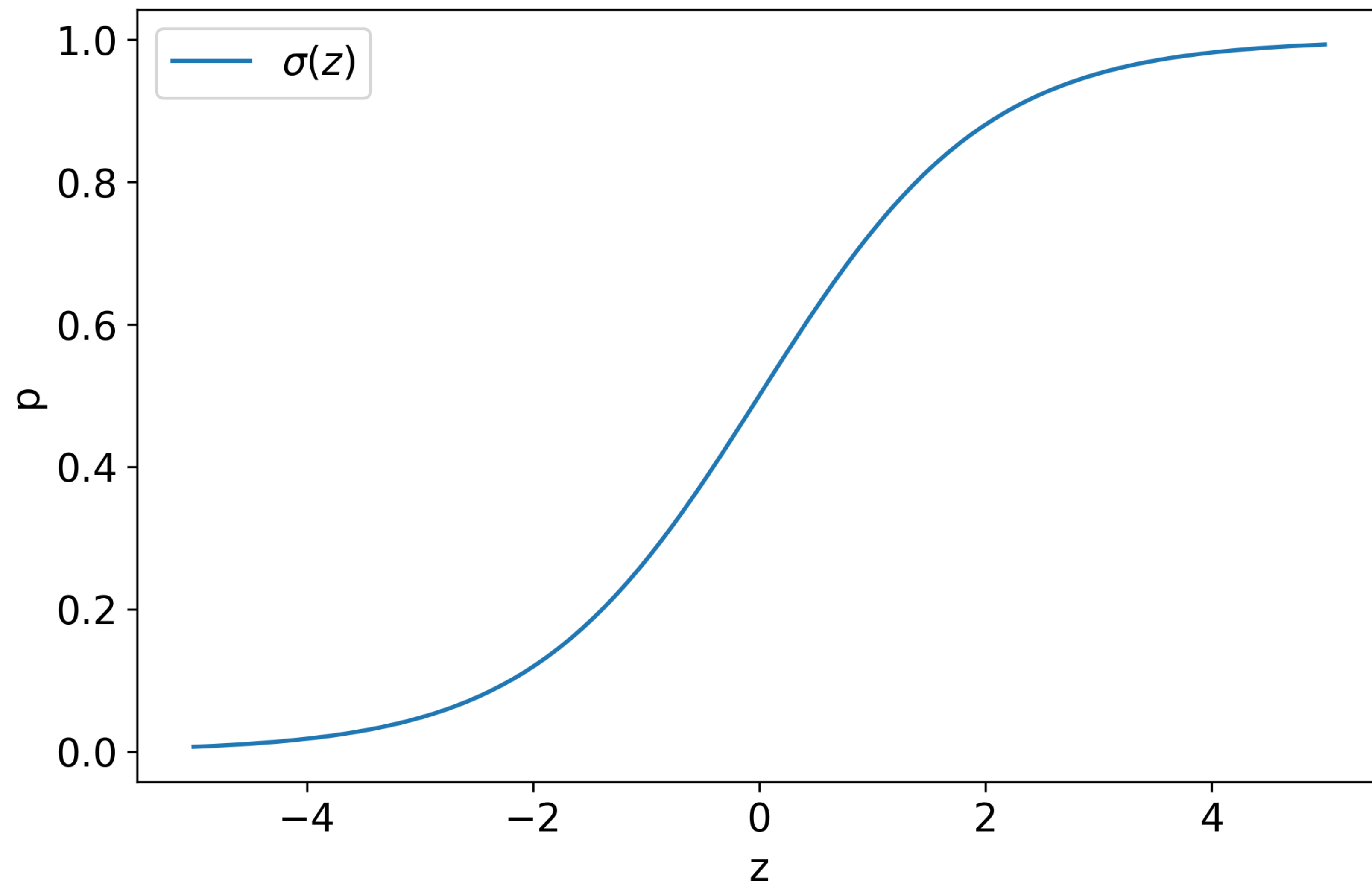
► for $(1-p)$:

$$1-p = \sigma(-z) \quad \frac{1+e^{-z}}{1+e^{-z}} - \frac{1}{1+e^{-z}} = \frac{e^{-z}}{1+e^{-z}} = \frac{1}{1+e^z}$$

- So $L = -Yz + \ln(1+e^z)$

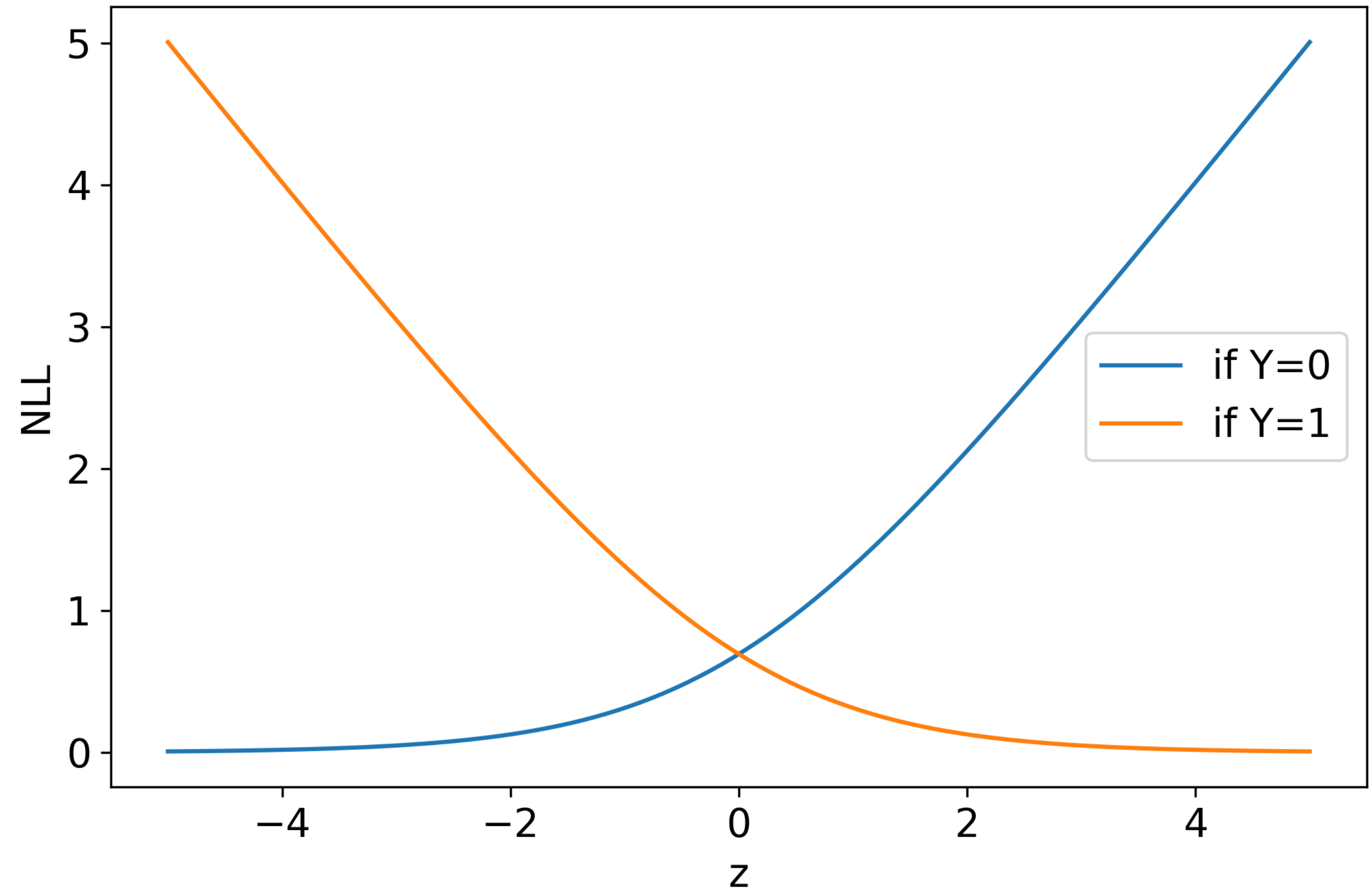
Probability as a function of log-odds

*called the **sigmoid** function
(means s-shaped)*



Bernoulli NLL as a function of log-odds

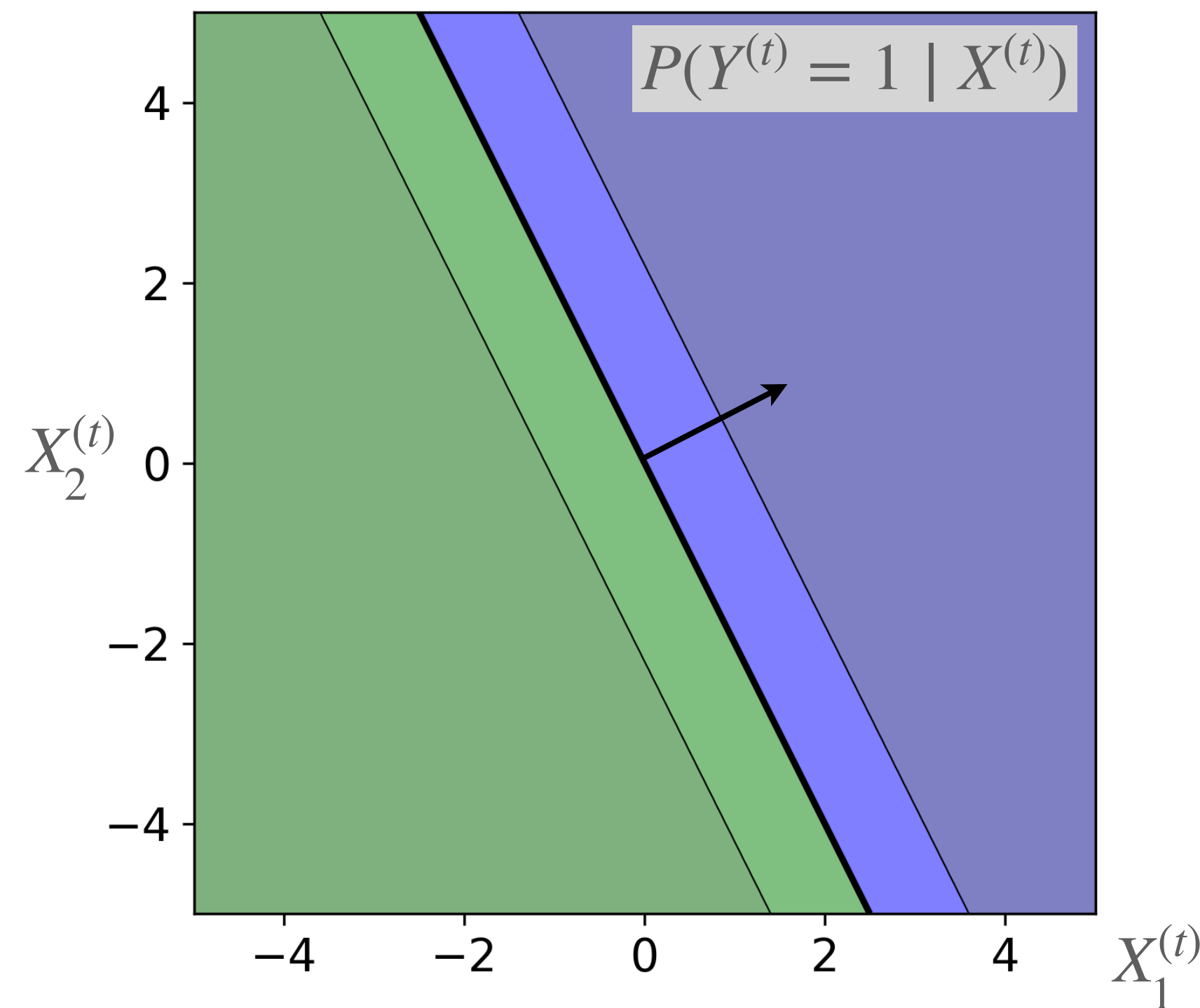
$$-y z + \ln(1 + e^z)$$



- Makes sense for any $z \in \mathbb{R}$
- Convex in z

Logistic regression

even though it's called "regression", this is a classifier learner



$$w = (2, 1)^T, \quad b = 0$$

*contours at probability
0.1, 0.5, 0.9*

*looks like a perceptron
w/ margin!*

- Instead of having $P(Y = 1 \mid X^{(t)})$ be linear in θ , make the **log-odds** be linear in θ given $X^{(t)}$
 - ▶ this makes the overall loss convex in θ (see HW; in general convex(linear) is convex)
- Model: $z^{(t)} = w \cdot X^{(t)} + b$ (or $\hat{Y}^{(t)} = \sigma(w \cdot X^{(t)} + b)$)
 - ▶ where log-odds for example t is $z^{(t)} = \ln \frac{\hat{Y}^{(t)}}{1 - \hat{Y}^{(t)}}$

Optimization

each term in sum is
convex, so we will
approach global
minimum

- NLL for logistic regression:

- ▶ $L = \sum_{t=1}^N [\ln(1 + \exp(z^{(t)})) - Y^{(t)} z^{(t)}]$

- ▶ where $z^{(t)} = X^{(t)} \cdot w + b$

- Differentiate the term for example t :

- ▶ $d \ln(1 + e^z) = \frac{1}{1 + e^z} e^z dz = \hat{y} dz$

- ▶ $dz = X^{(t)} \cdot dw + db$

- ▶ $dL = \left(\sum_{t=1}^N \right) \hat{y}^{(t)} (X^{(t)} \cdot dw + db) - \sum_{t=1}^N y^{(t)} (X^{(t)} \cdot dw + db)$
 $= (\hat{y}^{(t)} - y^{(t)}) (X^{(t)} \cdot dw + db)$

$$\frac{1}{1 + e^{-z}} = \sigma(z)$$

[Faint, illegible handwritten text]

take $\eta = 1$ for simplicity

$$\begin{aligned}
 &= X^{(+)} - 1 \\
 &- \cancel{\frac{1}{2}} X^{(+)} - \frac{1}{2} \\
 &\quad (\text{no update}) \\
 &+ \cancel{\frac{1}{2}} X^{(+)} + \frac{1}{2} \\
 &+ X^{(+)} \quad \quad \quad \cancel{\phi} 1
 \end{aligned}$$

- Geoff Gordon*

SGD for logistic regression

$$dL^{(t)} = (\hat{Y}^{(t)} - Y^{(t)})(X^{(t)} \cdot dw + db)$$

take $\eta = 1$ for simplicity

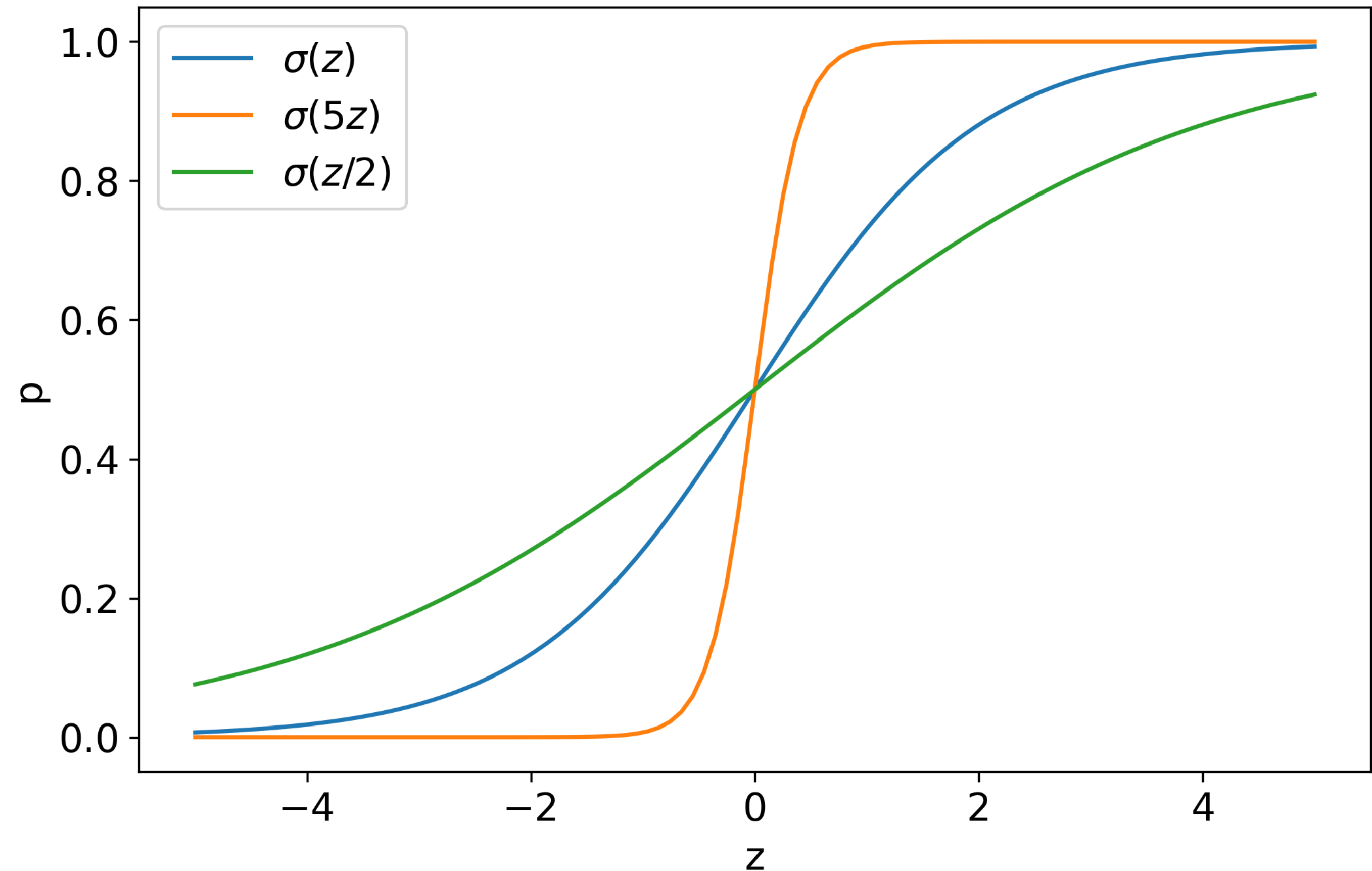
update to w to b

• If $\hat{Y}^{(t)} - Y^{(t)}$ is

- ▶ $+1$ (confident mistake on $-ve$): $\leftarrow +X^{(t)} \quad \leftarrow +1$
- ▶ $+\frac{1}{2}$ (predicted 50-50 on $-ve$): $\leftarrow +X^{(t)}/2 \quad \leftarrow +1/2$
- ▶ 0 (perfectly correct): (no update)
- ▶ $-\frac{1}{2}$ (predicted 50-50 on $+ve$): $\leftarrow -X^{(t)}/2 \quad \leftarrow -1/2$
- ▶ -1 (confident mistake on $+ve$): $\leftarrow -X^{(t)} \quad \leftarrow -1$

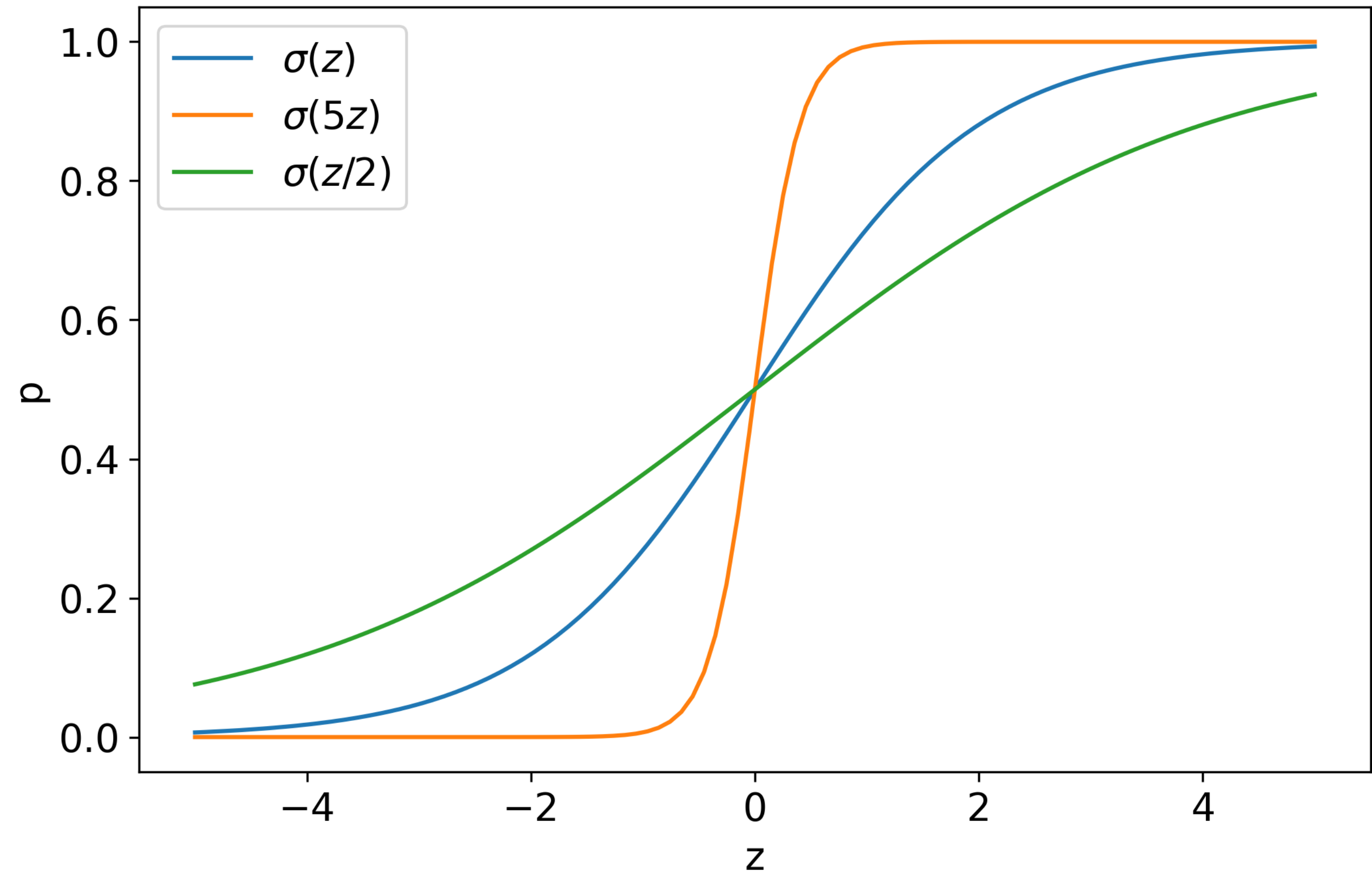
like a smoothed, probabilistic version of perceptron

Temperature



- Most of same derivation still holds if we use $\sigma(\beta z)$ instead of $\sigma(z)$ for any $\beta > 0$
 - ▶ $\frac{1}{\beta}$ is called *temperature* by analogy to statistical physics:
low temperature = reduced randomness

Temperature

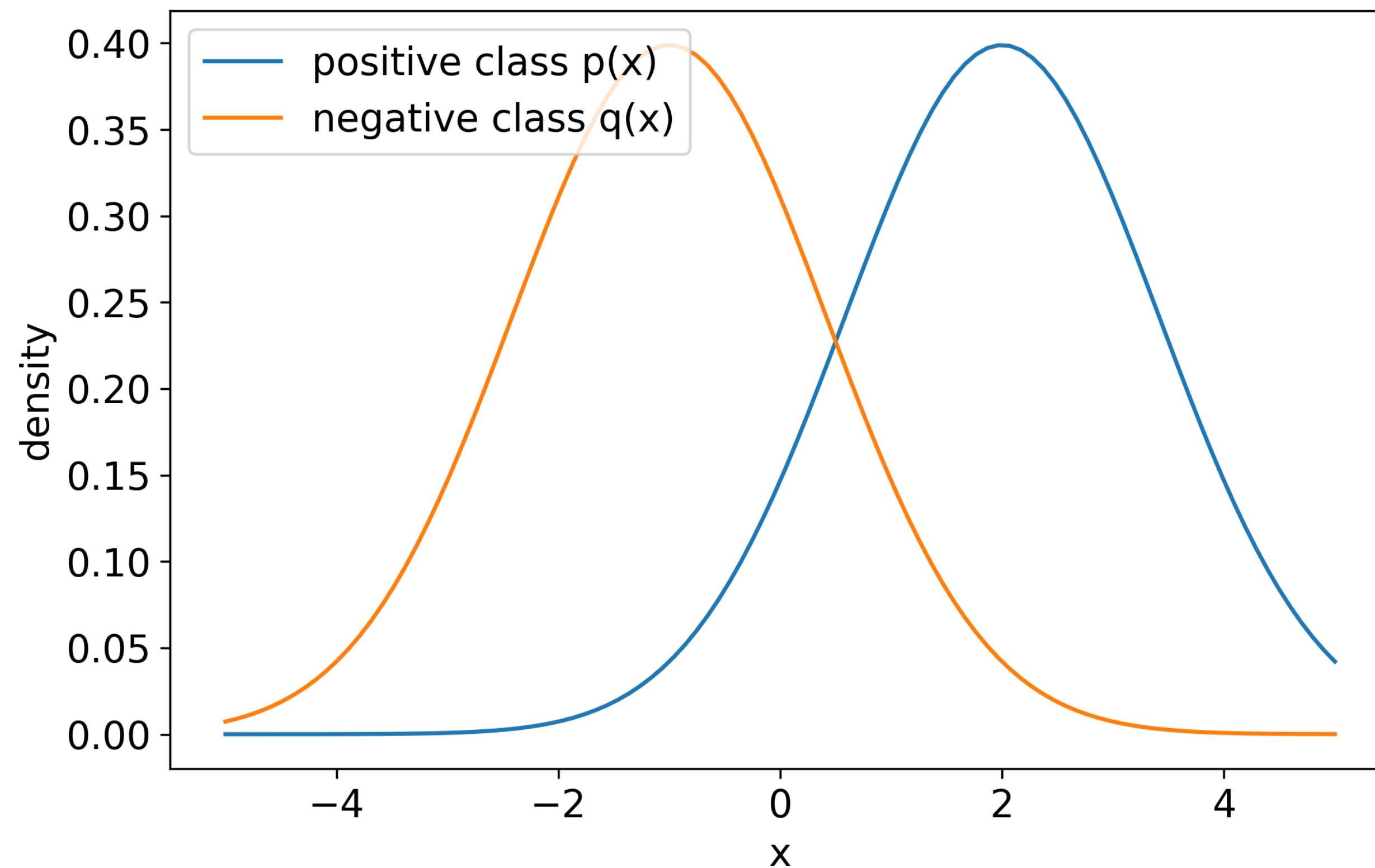


- MLE will compensate for temperature by scaling w, b , so this is most useful for adjustments after the fact
 - ▶ e.g., correct for overconfidence using validation set

Nonlinear

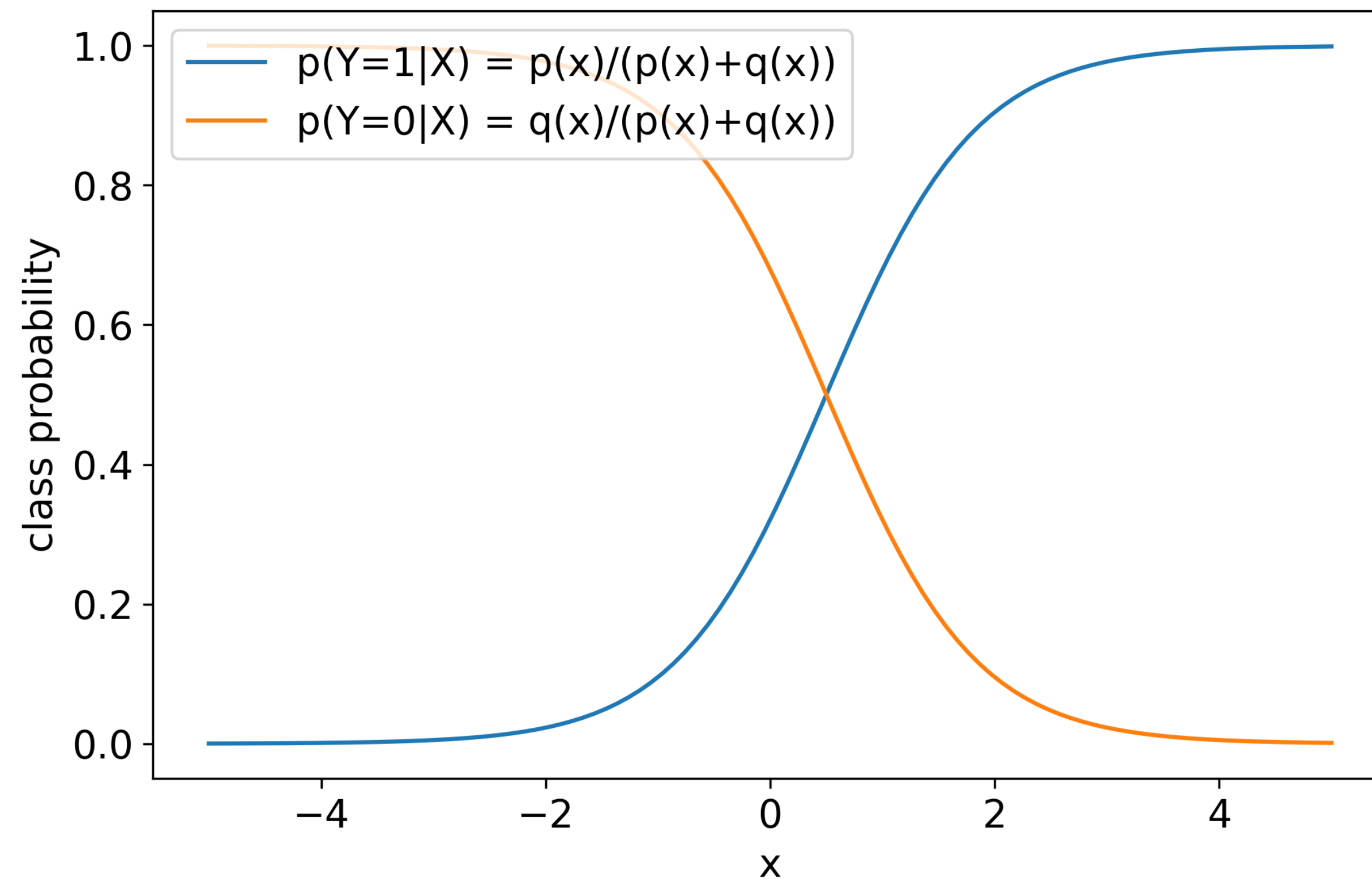
- Can also pick a nonlinear function class for log-odds:
 - ▶ $P(Y = 1 \mid X) = \sigma(f_\theta(X))$
 - ▶ $f_\theta = \text{MLP or transformer or ...}$
- In fact, this setup (sigmoid nonlinearity and Bernoulli log-likelihood loss) is one of the most common last layers for neural network classification
- NLL is still convex in predicted log-odds $z = f_\theta(X)$, but typically nonconvex in θ

Bayes optimal classifier



- Usually we don't model $P(X)$
- But suppose we included it in generative story:
 - ▶ flip a coin to decide whether to generate a positive or negative example
 - ▶ if positive, $X^{(t)} \sim p(X)$, $Y^{(t)} = 1$
 - ▶ if negative, $X^{(t)} \sim q(X)$, $Y^{(t)} = 0$

Bayes optimal classifier



- If we see a new example X
 - ▶ prior: $P(Y = 0) = P(Y = 1) = \frac{1}{2}$
 - ▶ posterior: $P(Y = 1 | X) = P(X | Y = 1) P(Y = 1) / Z$
 - ▶ and $P(Y = 0 | X) =$

$$\frac{p(x)}{p(x) + q(x)}$$

$$\frac{q(x)}{p(x) + q(x)}$$

$$\frac{p(x)}{q(x)} \cdot \frac{\frac{1}{2}}{\frac{1}{2}} / Z$$

Cf. logistic regression

- Instead of learning $P(Y | X)$, learn $P(X | Y)$ and use Bayes rule to convert:
 - ▶ write $p(X) = P_{\theta}(X | Y = 1)$ and $q(X) = P_{\phi}(X | Y = 0)$
$$P(Y = 1 | X) = \frac{p(X)}{p(X) + q(X)} \quad P(Y = 0 | X) = \frac{q(X)}{p(X) + q(X)}$$
 - ▶ or log-odds $z = \ln \frac{p(X)}{q(X)} = f_{\theta, \phi}(X) \leftarrow \text{could be linear or nonlinear}$
 - ▶ and prediction $\hat{Y} = \sigma(z)$
- Same overall idea (learn parameters of a discriminant function that gives us log-odds) but
 - ▶ different way to get there
 - ▶ possibly different functional form
 - ▶ possibly different optimal parameters

Gaussian Bayes

a linear classifier —
same as logistic
regression for big N if
 p, q are truly Gaussian
— different in general

closed-form, no need
for SGD

- With Gaussian class-conditional distributions:

- ▶ $P(X | Y = 1) = p(X) = N(\mu_1, \Sigma)$

- ▶ $P(X | Y = 0) = q(X) = N(\mu_0, \Sigma)$

- ▶ with $N(\mu, \Sigma) = \frac{1}{\sqrt{|2\pi\Sigma|}} \exp(-\frac{1}{2}(X - \mu)^T \Sigma^{-1}(X - \mu))$

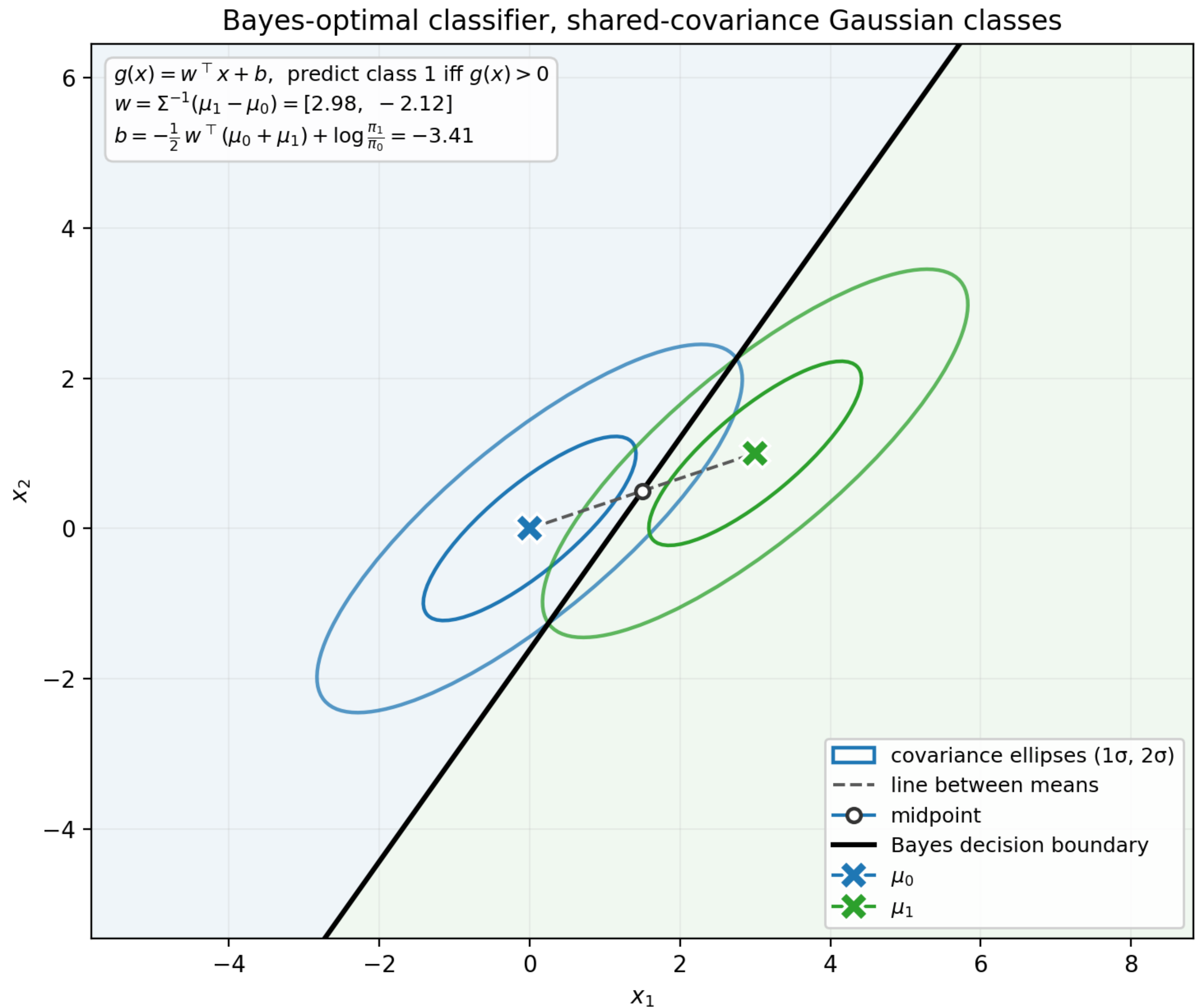
- ▶ $\ln \frac{p(X)}{q(X)} =$

$$-\frac{1}{2} (X^T \Sigma^{-1} X - 2X^T \Sigma^{-1} \mu_1 + \mu_1^T \Sigma^{-1} \mu_1)$$

$$+ \frac{1}{2} (X^T \Sigma^{-1} X - 2X^T \Sigma^{-1} \mu_0 + \mu_0^T \Sigma^{-1} \mu_0)$$

$$\underbrace{X^T \Sigma^{-1} (\mu_0 - \mu_1)}_w - \underbrace{w^T \left(\frac{\mu_0 + \mu_1}{2} \right)}_{b}$$

Gaussian Bayes



Gaussian naive Bayes

*same as logistic
regression for big N if
 p, q are truly diagonal
Gaussian— different in
general*

*closed-form, no need
for SGD*

- Simple version of Gaussian: $\Sigma = \text{diag}(\sigma_1^2, \dots, \sigma_d^2)$, independent features
 - ▶ classifier called **naive** Bayes for simplifying assumption
- From training set:
 - ▶ for each feature j , find μ_j^+ , μ_j^- , and σ_j^2
$$\sigma_j^2 = \frac{1}{N-1} \left[\sum_{t \in X^+} (X_j^{(t)} - \mu_j^+)^2 + \sum_{t \in X^-} (X_j^{(t)} - \mu_j^-)^2 \right] \quad \text{pooled variance}$$
 - ▶ store $w_j = \frac{1}{\sigma_j^2}(\mu_j^+ - \mu_j^-)$, $b_j = -\frac{1}{2}(\mu_j^+ + \mu_j^-) w_j$
- For a test point X :
 - ▶ sum up evidence for each feature: $z = \sum_j (w_j X_j + b_j)$
 - ▶ predict $\sigma(z)$

actual samples (minor edits for privacy)

From: no-reply@aksdfn2dk4.net

To: ggordon@cmu.edu

*WE'RE GIVING YOU AN EXCLUSIVE
OPPORTUNITY TO RECEIVE A
BRAND-NEW YYYYYY Bundle. This
exclusive, limited-time reward
won't be available for long.*

From: no-reply@piazza.com

To: ggordon@cmu.edu

*A new question was posted by
XXXXXXX.*

*I had a question about the
derivation for homework
question 2.3.*

Categorical naive Bayes

*often just called
"naive Bayes"*

- Can use same independence assumption for categorical features (not actually independent, but good enough)

- ▶ e.g, phrase “exclusive opportunity”, or from-address “no-reply@piazza.com”, or use of all-caps

- ▶ $P(Y = y | X) = P(X | Y = y)P(Y = y)/Z \quad (y \in \{0,1\})$

- ▶ $P(X | Y = y) = \prod_k P(X_k | Y = y)$

- ▶ so $z = \ln \frac{P(Y = 1 | X)}{P(Y = 0 | X)} = \ln \frac{P(Y=1)}{P(Y=0)} + \sum_k \ln \frac{P(X_k | Y=1)}{P(X_k | Y=0)}$

- ▶ e.g., ALL-CAPS:

*5% of 1 ln 5
10% of 0*

Multiclass

- So far, two classes (+ and -)
- Can extend logistic regression to work with any number of classes
- Most common parameterization: one linear function per class, $z_k = w_k \cdot X + b_k$ (nonlinear extension also works)

▶ z_k is still called a logit

▶ represents $\ln P(Y = k | X) + \text{const}$

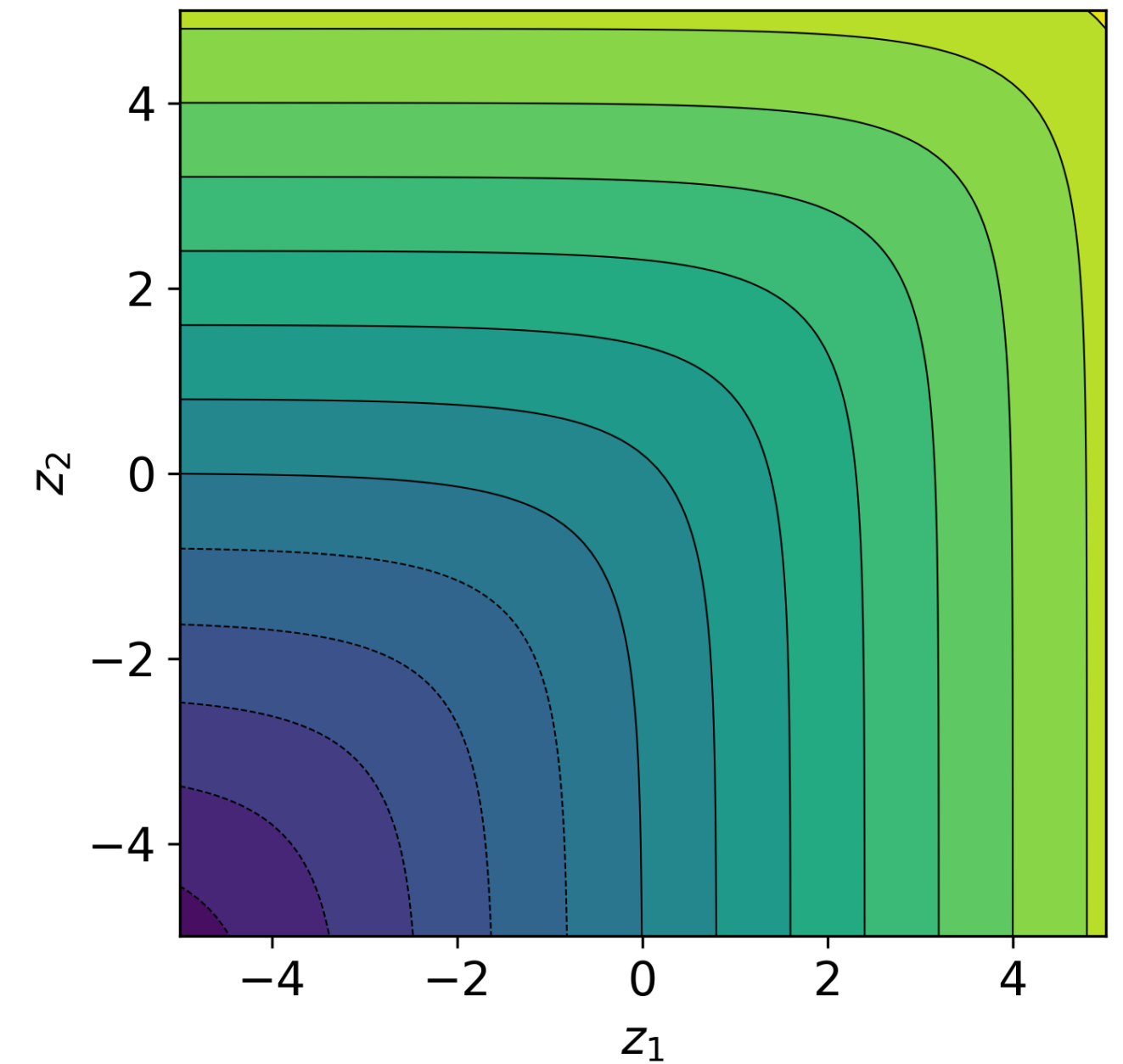
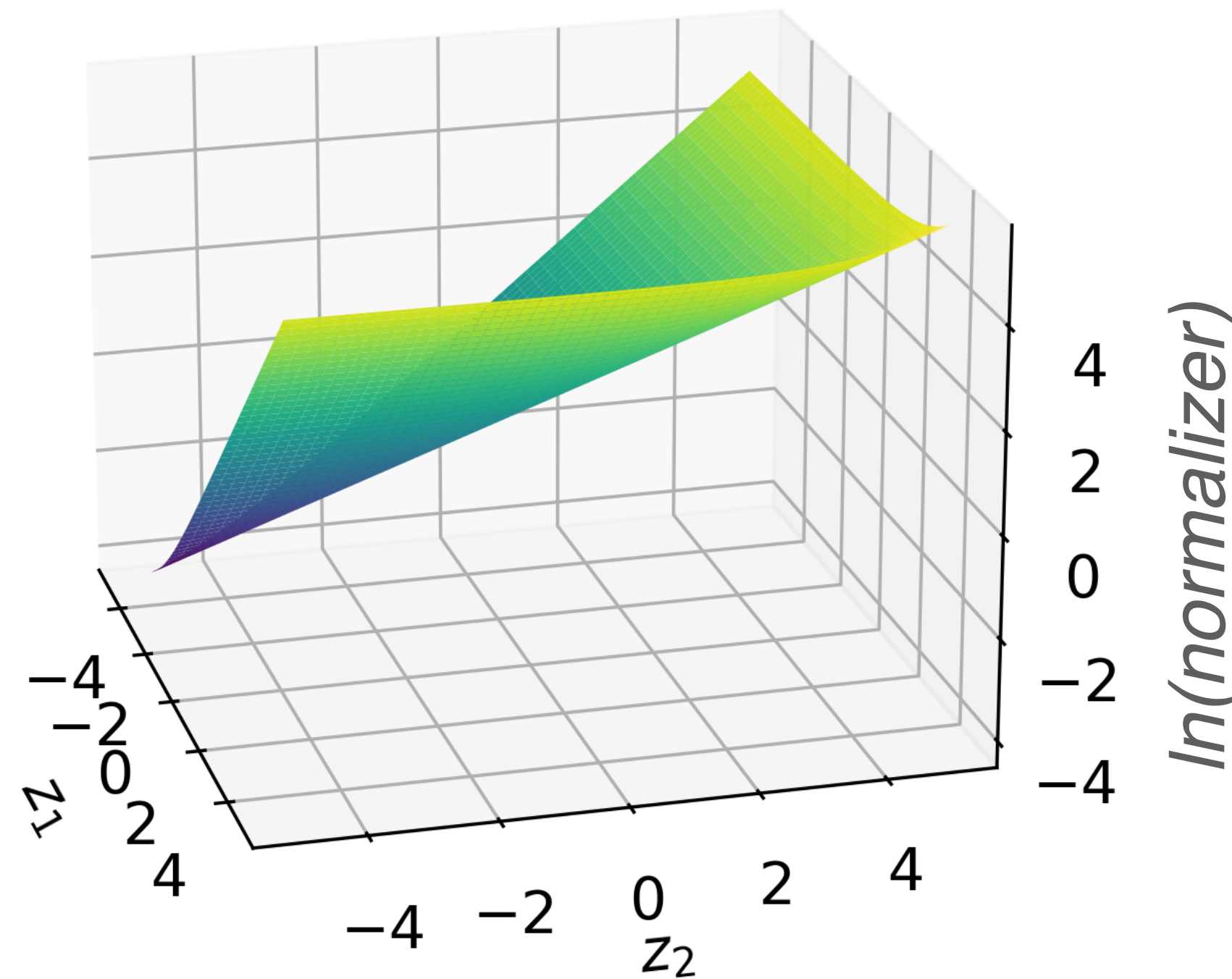
▶ retrieve $\hat{Y}_k = P(Y = k | X)$ as $\frac{\exp(z_k)}{\sum_{k'} \exp(z_{k'})} =$

called the **softmax** function,
multidimensional
version of sigmoid

$$\frac{e^{\ln P(Y=k|X) + C}}{\sum_{k'} e^{\ln P(Y=k'|X) + C}} = \frac{P(Y=k|X) e^C}{\sum_{k'} P(Y=k'|X) e^C} = \frac{P(Y=k|X)}{\sum_{k'} P(Y=k'|X)} = 1$$

Cross-entropy loss

if you need to calculate $\ln(\text{sum}(\exp))$, beware of overflow



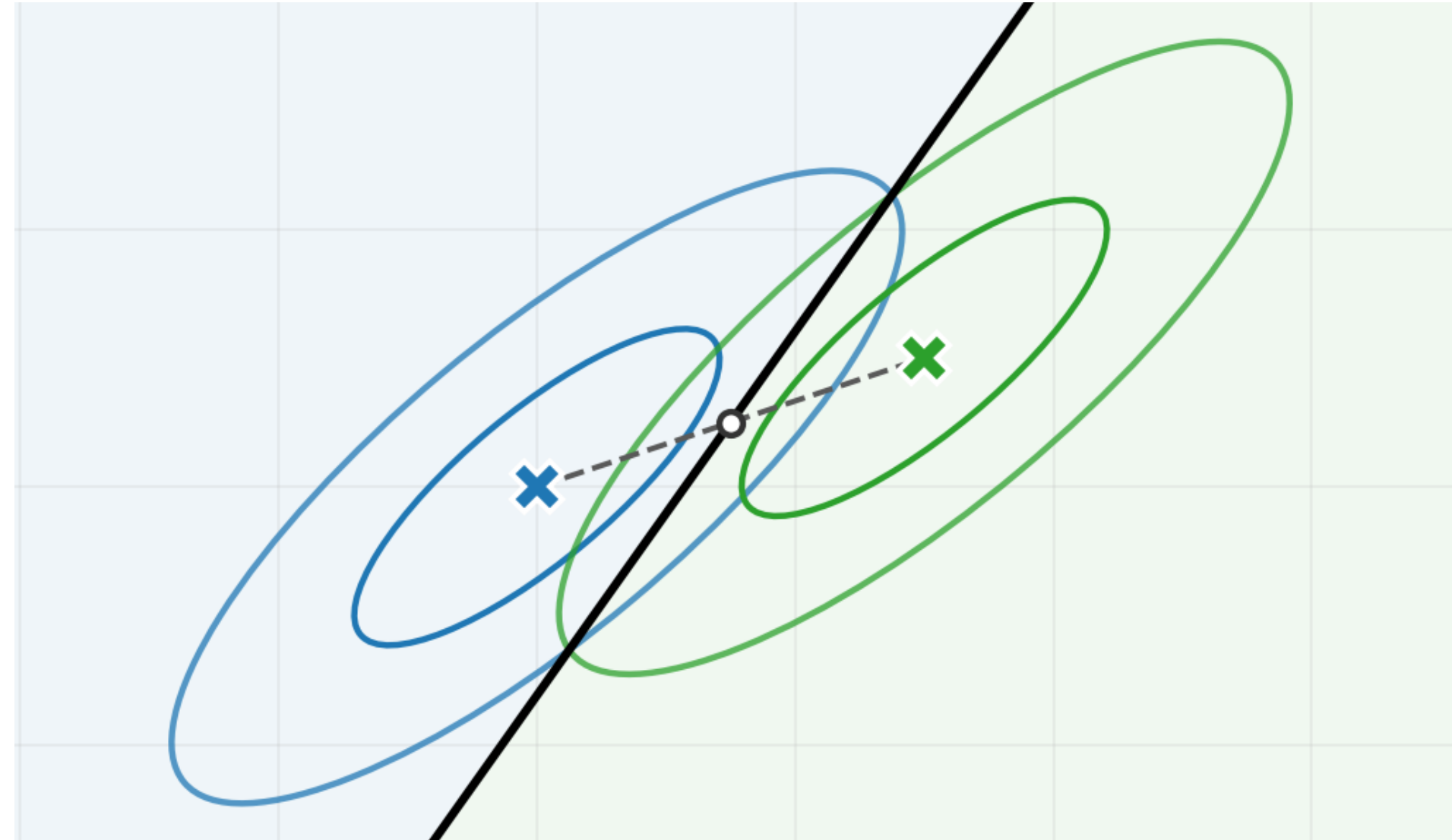
- Multiclass version of Bernoulli is **categorical** distribution
 - ▶ NLL is $-\sum_k Y_k \ln \hat{Y}_k$, the **cross-entropy**
 - ▶ where $\hat{Y}_k = \frac{\exp(z_k)}{\sum_{k'} \exp(z_{k'})}$ is predicted probability of class k
- Softmax w/ cross-entropy is a **convex** function of logits z

$$-\sum_k Y_k \ln \hat{Y}_k = -\sum_k Y_k \left[z_k - \ln \sum_{k'} \exp(z_{k'}) \right] = \ln \sum_{k'} \exp(z_{k'}) - \sum_k Y_k z_k$$

Strong relationship between classifiers and distributions

- We showed how to
 - ▶ learn a classifier (logistic regression, max conditional likelihood)
 - ▶ learn a distribution (MLE)
 - ▶ learn a classifier by learning distributions (Bayes and naive Bayes)
 - ▶ ...learn a distribution by learning classifiers (???)

Discriminative learning



- Suppose we have a model of a distribution q (and can draw samples from it)
- Suppose we have samples from another distribution p
- Learn to classify samples from p vs samples from q

▶ optimal discriminant: $z(x) = \ln \frac{p(x)}{q(x)}$

- Now consider $q(x)e^{z(x)} = \cancel{q(x)} \frac{p(x)}{\cancel{q(x)}} = p(x)$

Pros and cons

- Pro: in high-d, it can be hard to learn a model of $p(X)$ directly; classifiers can be easier
- Cons:
 - ▶ might be hard to work with $q(x)e^{z(x)}$ (normalize, draw samples)
 - ▶ might be hard to learn anything close to optimal classifier, especially if p is far from q
 - ▶ sanity check: ratio of two distributions that are uniform in some area $A \rightarrow$ accuracy at most $\lceil \log_2(N + 1) \rceil$ bits after N samples hit A , since there are only $N + 1$ outcomes for empirical class ratio